ESTRUCTURA DEL PROGRAMA:

HEADERS:

Los headers son archivos hechos en lenguaje C, que para el funcionamiento de este proyecto fueron usados para manejar la estructura de datos que usa el programa al momento de darle lectura a los archivos en formato lfp. Su principal función es darle acceso a las demás clases al momento de organizar cada clase para el buen funcionamiento del programa.

Curso.h

Dentro de este header, pondremos la estructura del curso:

Código, créditos, semestre como datos de tipo entereo, nombre y carrera siendo una cadena de texto.

```
struct Curso {
    int codigo;
    string nombre;
    int creditos;
    int semestre;
    string carrera;
};
```

También agregaremos un vector para la carga de cursos solicitando la ruta del archivo, y el método de creación de cursos solicitando la línea inicial.

```
Curso crearCurso(const string& linea);
vector<Curso> cargarCursos(const string& ruta);
```

clase nota

proceso similar, variando datos como carnet, codigocurso, nota, ciclo y el año(usamos anio debido a restricciones de c++).

```
Nota crearNota(const string& linea);
vector<Nota> cargarNotas(const string& ruta);
```

```
struct Nota {
    int carnet;
    int codigoCurso;
    double nota;
    string ciclo;
    int anio;
};
```

3. Métodos principales

- **Carga de datos**
  - cargarEstudiantes(ruta)
  - cargarCursos(ruta)
  - cargarNotas(ruta)
- **Procesamiento estadístico**
  - promedio(vector)
  - mediana(vector)
  - desviacionEstandar(vector)
- **Generación de reportes**

- reporteCursos(cursos, notas)
- reporteEstudiantes(estudiantes, notas, cursos)
- reporteTop10(estudiantes, notas)
- reporteReprobacion(cursos, notas)
- reporteCarreras(estudiantes, cursos, notas)

4. Requerimientos técnicos

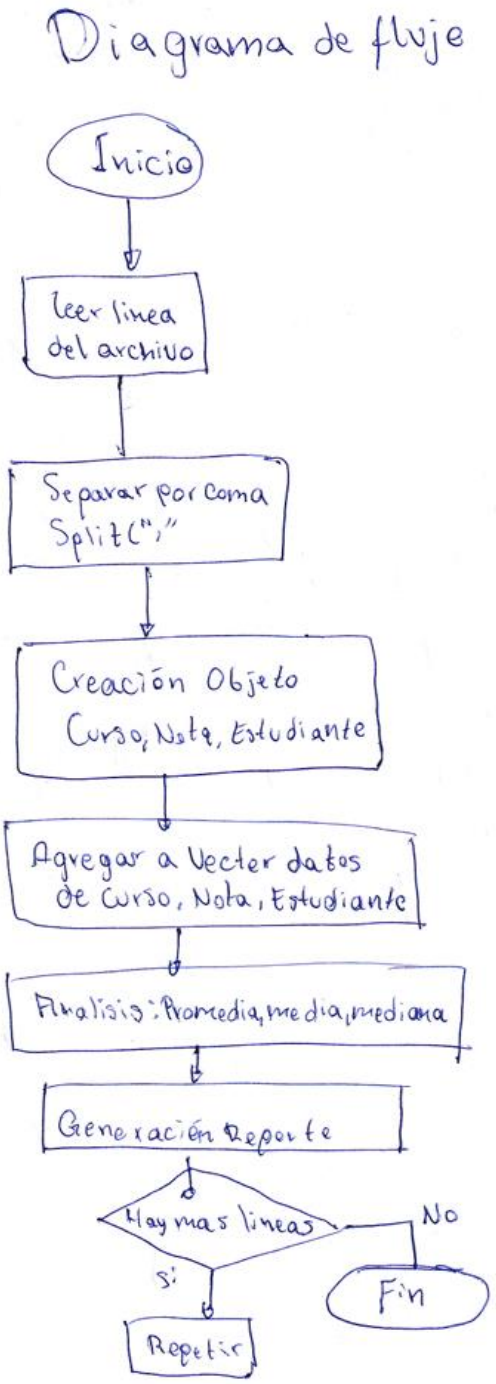
- **Lenguaje:** C++17 o superior
- **Compilador:** g++ (MinGW en Windows, GCC en Linux/Mac)
- **Dependencias:** STL (, , , , ,)
- **Sistema operativo:** Windows, Linux o MacOS
- **Navegador web:** Para visualizar los reportes HTML

5. Lógica de lectura y procesamiento de archivos

Pseudocódigo de carga de cursos

```
función cargarCursos(ruta):
    abrir archivo en modo lectura
    si no se puede abrir:
        mostrar error y retornar lista vacía
    mientras haya líneas en el archivo:
        dividir línea con split(',')
        crear objeto Curso con los campos
        agregar Curso a la lista
    cerrar archivo
    retornar lista de cursos
```

Diagrama de flujo simplificado



6. Observaciones relevantes

- Los archivos .lfp deben estar bien formateados con campos separados por comas.
- La función split centraliza la lógica de separación de campos y se reutiliza en Estudiante.cpp, Curso.cpp y Nota.cpp.
- Los reportes se generan en HTML dentro de la carpeta reportes/ y pueden abrirse en cualquier navegador.
- El menú principal (menu.cpp) mantiene los vectores cargados en memoria y permite generar reportes sin reiniciar el programa.
- Se recomienda validar datos con try/catch para evitar errores en stoi si los archivos contienen valores inválidos.

⟐ 7. Ejemplo de ejecución

Consola

```
cout << "SISTEMA DE ANALISIS ACADEMICO" << endl;
        cout << "1. Cargar archivo de estudiante" << endl;
        cout << "2. Cargar archivo de notas " << endl;
        cout << "3. Cargar archivo de curso" << endl;
        cout << "4. Generar Reporte: Estadisticas por Curso" << endl;
        cout << "5. Generar Reporte: Rendimiento por Estudiante" << endl;
        cout << "6. Generar Reporte: Top 10 Mejores Estudiantes" << endl;
        cout << "7. Generar Reporte: Cursos con Mayor Reprobacion" << endl;
        cout << "8. Generar Reporte: Analisis por Carrera" << endl;
        cout << "9. Salir" << endl;
        cout << "Seleccione una opcion" << endl;
```

Reporte HTML (ejemplo simulado)

Estadísticas por Curso

Curso	Estudiantes	Promedio
Matemáticas	35	74.5
Física	28	68.2

Este sistema permite **cargar datos académicos**, **procesarlos estadísticamente** y **generar reportes HTML** de manera modular y clara. La organización en carpetas (include/, src/, reportes/) asegura mantenibilidad y escalabilidad del proyecto.